ORDERS

FROM

JOIN

WHERE

GROUP BY

HAVING

ORDER BY

LIMIT

SELECT

Common Mistakes After Where (“in” vs “=”)

Common Mistakes Where vs Having

Common Mistakes Max () vs Limit

Join works row by row whereas ‘in’ checks only existence or simply makes a set

‘-’= ‘except’, U= ‘Union’, last one Intersection

Subquery Examples with Solutions

1. Find customers who placed orders

```
SELECT first_name, last_name
FROM Customers
WHERE customer_id IN (
   SELECT customer_id
   FROM Orders
);
```

- Inner query returns customer IDs from Orders
- Outer query finds matching customers

John Doe
Robert Luna
David Robinson
John Reinhardt

2. Find customers who have NOT placed any order

```
SELECT first_name, last_name
FROM Customers
WHERE customer_id NOT IN (
   SELECT customer_id
   FROM Orders
);
```

Betty Doe

3. Find customers whose order amount is greater than average order amount

```
SELECT first_name, last_name
FROM Customers
WHERE customer_id IN (
   SELECT customer_id
   FROM Orders
   WHERE amount > (
      SELECT AVG(amount) FROM Orders
   )
);
```

- First subquery calculates average order amount
- Second subquery finds customers with higher orders

4. Find customers whose shipping status is Delivered

```
SELECT first_name, last_name
FROM Customers
WHERE customer_id IN (
   SELECT customer
   FROM Shippings
   WHERE status = 'Delivered'
);
```

David Robinson
John Doe

5. Find customers who ordered Keyboard

```
SELECT first_name, last_name
FROM Customers
WHERE customer_id IN (
   SELECT customer_id
   FROM Orders
   WHERE item = 'Keyboard'
);
```

John Doe
John Reinhardt

6. Find customers older than average age

```
SELECT first_name, last_name, age
FROM Customers
WHERE age > (
   SELECT AVG(age)
   FROM Customers
);
```

John Doe
Betty Doe

7. Find customers who have pending shipping

```
SELECT first_name, last_name
FROM Customers
WHERE customer_id IN (
   SELECT customer
   FROM Shippings
   WHERE status = 'Pending'
);
```

Robert Luna
John Reinhardt
Betty Doe

8. Find the customer who placed the highest order amount

```
SELECT first_name, last_name
FROM Customers
WHERE customer_id IN (
   SELECT customer_id
   FROM Orders
   ORDER BY amount DESC
   LIMIT 1
);
```

first_name	last_name
David	Robinson

9. Find the customer who placed the second highest order

```
SELECT first_name, last_name
FROM Customers
WHERE customer_id = (
   SELECT customer_id
   FROM Orders
   ORDER BY amount DESC
   LIMIT 1 OFFSET 1
);
```

first_name	last_name
John	Reinhardt

10. Find the Youngest Customer

```
SELECT *
FROM Customers c
WHERE c.age = (
   SELECT MIN(age)
   FROM Customers
);
```

customer_id	first_name	last_name	age	country
2	Robert	Luna	22	USA
3	David	Robinson	22	UK

Alternative

```
SELECT *
FROM Customers
ORDER BY age ASC
LIMIT 1;
```

⚠ **This version returns only** one row**,** even if multiple customers share the same minimum age.

Don’t read the read colored documents

11. Customers who placed orders

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    o.item,
    o.amount
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id;
```

12. LEFT JOIN – All customers (with or without orders)

```
SELECT
    c.customer_id,
    c.first_name,
    o.item,
    o.amount
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id;
```

13. Customers with shipping status

```
SELECT
    c.customer_id,
    c.first_name,
    s.status
FROM Customers c
INNER JOIN Shippings s
ON c.customer_id = s.customer;
```

14. Customers + Orders + Shipping (FULL DETAILS)

```
SELECT
    c.customer_id,
    c.first_name,
    o.item,
    o.amount,
    s.status
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
LEFT JOIN Shippings s
ON c.customer_id = s.customer;
```

15. Customers who have NO orders

```
SELECT
   c.customer_id,
   c.first_name
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

16. Customers who ordered AND shipping is Pending

```
SELECT
   c.customer_id,
   c.first_name,
   o.item,
   s.status
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
JOIN Shippings s
ON c.customer_id = s.customer
WHERE s.status = 'Pending';
```

17. Total amount spent by each customer

```
SELECT
   c.customer_id,
   c.first_name,
   SUM(o.amount) AS total_spent
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name;
```

Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

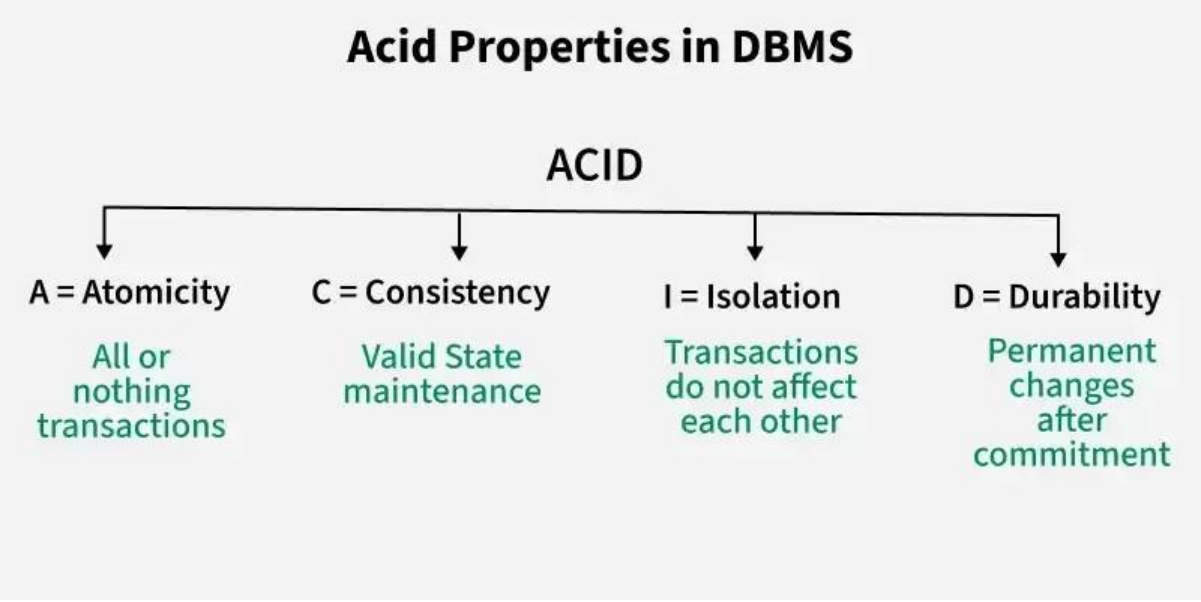
Shippings

shipping_id	status	customer
1	Pending	2
2	Pending	4
3	Delivered	3
4	Pending	5
5	Delivered	1

ACID Properties in DBMS

Transactions are fundamental operations that allow us to modify and retrieve data. However, to ensure the integrity of a database, it is important that these transactions are executed in a way that maintains consistency, correctness, and reliability even in case of failures / errors. This is where the ACID properties come into play.

ACID stands for Atomicity, Consistency, Isolation, and Durability.



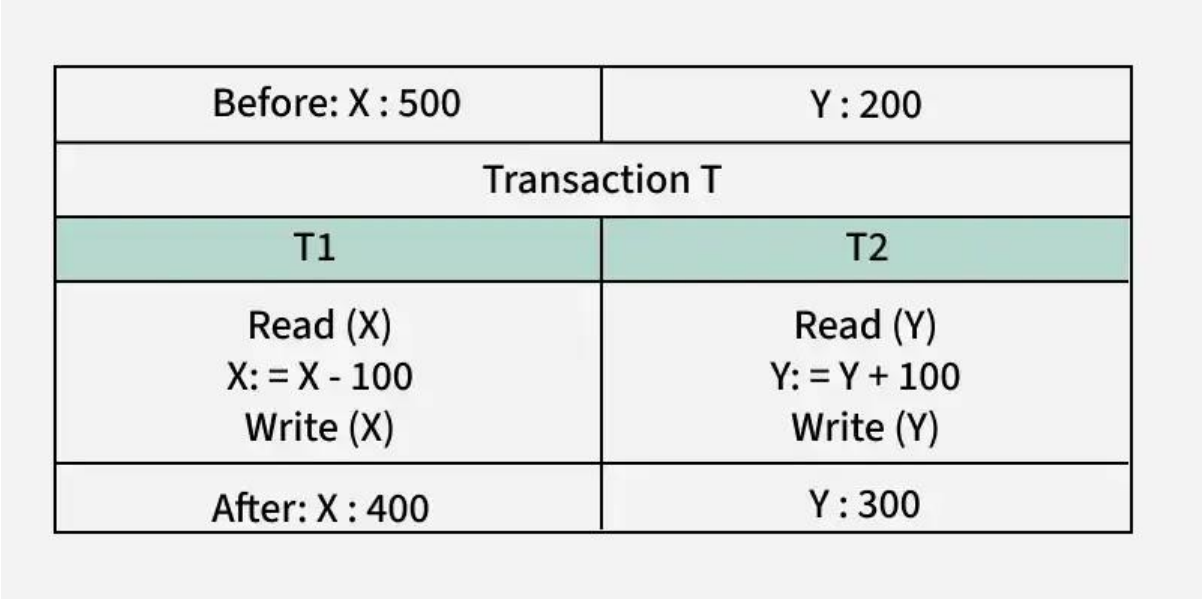
There are Four Properties of ACID

1. **Atomicity**

Atomicity means a transaction is all-or-nothing either all its operations succeed, or none are applied. If any part fails, the entire transaction is rolled back to keep the database consistent.

- **Commit**: If the transaction is successful, the changes are permanently applied.
- **Abort/Rollback**: If the transaction fails, any changes made during the transaction are discarded.

Example: Consider the following transaction **T** consisting of **T1** and **T2** : Transfer of $100 from account **X** to account **Y** .



Before: X : 500	Y : 200
Transaction T	
T1	T2
Read (X) X: = X - 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

Atomicity

If the transaction fails after completion of T1 but before completion of T2, the database would be left in an inconsistent state. W ith Atomicity, if any part of the transaction fails, the entire process is rolled back to its original state, and no partial changes are made.

2. Consistency

Consistency in transactions means that the database must remain in a valid state before and after a transaction.

- A valid state follows all defined rules, constraints, and relationships (like primary keys, foreign keys, etc.).
- If a transaction violates any of these rules, it is rolled back to prevent corrupt or invalid data.
- If a transaction deducts money from one account but doesn't add it to another (in a transfer), it violates consistency.

Example: Suppose the sum of all balances in a bank system should always be constant. Before a transfer, the total balance is $700. After the transaction, the total balance should remain $700. If the transaction fails in the middle (like updating one account but not the other), the system should maintain its consistency by rolling back the transaction.

Total before T occurs = 500 + 200
= 700 . **Total after T occurs** = 400
+ 300 = 700 .

X = 500 Rs | Y = 500 Rs

T	T''
Read (X) X :=X*100 Write (X) Read (Y) Y:= Y - 50 Write (Y)	Read (X) Read (Y) Z:= X + Y Write (Z)

Consistency

3. Isolation

Isolation ensures that transactions run independently without affecting each other. Changes made by one transaction are not visible to others until they are committed.

It ensures that the result of concurrent transactions is the same as if they were run one after another, preventing issues like:

- **Dirty reads:** reading uncommitted data
- **Non-repeatable reads:** data changes between two reads
- **Phantom reads:** new rows appear during a transaction

Example: Consider two transactions T and T".

- **X = 500, Y = 500**

X = 500 Rs	Y = 500 Rs
T	T''
Read (X) X := X - 50 Write (X) Read (Y) Y:= Y - 50 Write (Y)	Read (X) Read (Y) Z:= X + Y Write (Z)

Isolation

Explanation:

1. **Transaction T:**

- T wants to transfer $50 from X to Y.
- T reads **Y** (value: 500), deducts $50 from X (new X = 450), and adds $50 to Y (new Y = 550).

2. **Transaction T'':**

- T'' starts and reads X (500) and Y (500).
- It calculates the sum: 500 + 500 = 1000.
- Meanwhile, values of X and Y change to 450 and 550 respectively.
- So, the correct sum should be 450 + 550 = 1000.
- Isolation ensures that T'' does not read outdated values while another transaction (T) is still in progress.
- Transactions should be independent, and T'' should access the final values only after T commits.
- This avoids inconsistent results, like the incorrect sum calculated by T''.

4. Durability:

Durability ensures that once a transaction is committed, its changes are permanently saved, even if the system fails. The data is stored in non-volatile memory, so the database can recover to its last committed state without losing data.

Example: After successfully transferring money from Account A to Account B, the changes are stored on disk. Even if there is a crash immediately after the commit, the transfer details will still be intact when the system recovers, ensuring durability.

How ACID Properties Impact DBMS Design and Operation

The ACID properties, in totality, provide a mechanism to ensure the correctness and consistency of a database in a way such that each transaction is a group of operations that acts as a single unit, produces consistent results, acts in isolation from other operations, and updates that it makes are durably stored.

1. Data Integrity and Consistency

ACID properties safeguard the data integrity of a DBMS by ensuring that transactions either complete successfully or leave no trace if interrupted. They prevent partial updates from corrupting the data and ensure that the database transitions only between valid states.

2. Concurrency Control

ACID properties provide a solid framework for managing concurrent transactions. Isolation ensures that transactions do not interfere with each other, preventing data anomalies such as lost updates, temporary inconsistency, and uncommitted data.

3. Recovery and Fault Tolerance

Durability ensures that even if a system crashes, the database can recover to a consistent state. Thanks to the Atomicity and Durability properties, if a transaction fails midway, the database remains

in a consistent state.

Property	Responsibility for maintaining properties
Atomicity	Transaction Manager
Consistency	Application programmer
Isolation	Concurrency Control Manager
Durability	Recovery

Critical Use Cases for ACID in Databases

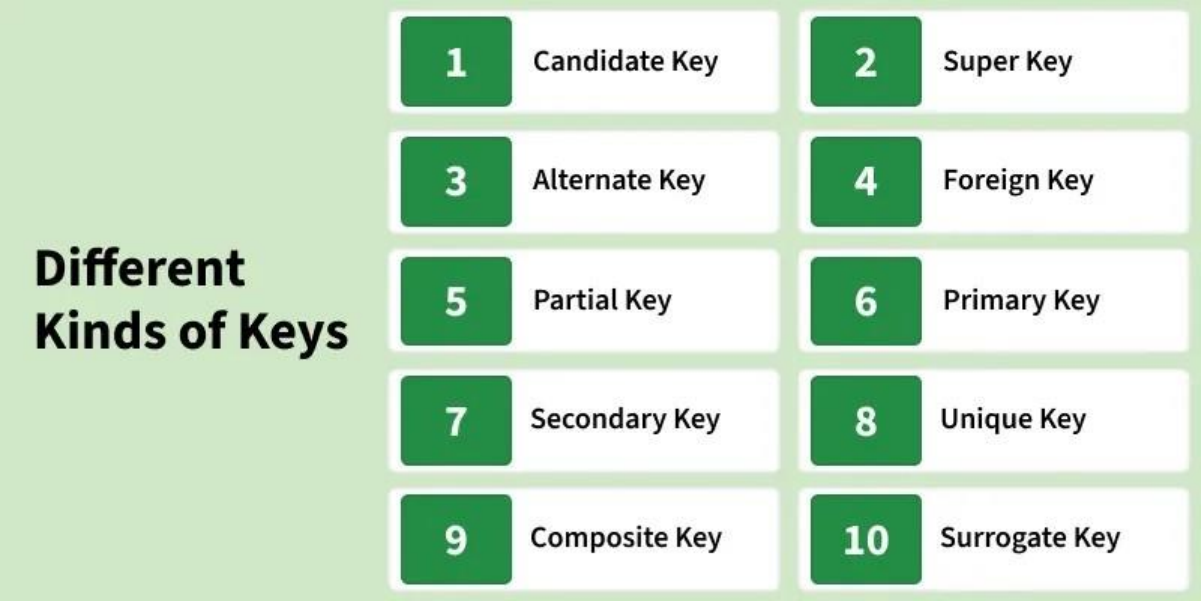
In modern applications, ensuring the reliability and consistency of data is crucial. ACID properties are fundamental in sectors like:

- **Banking**: Transactions involving money transfers, deposits, or withdrawals must maintain strict consistency and durability to prevent errors and fraud.
- **E-commerce**: Ensuring that inventory counts, orders, and customer details are handled correctly and consistently, even during high traffic, requires ACID compliance.
- **Healthcare**: Patient records, test results, and prescriptions must adhere to strict consistency, integrity, and security standards.

Keys in Relational Model

Keys are fundamental elements of the relational database model that ensure uniqueness, data integrity, and efficient data access.

- They uniquely identify each row in a table.
- They prevent data duplication and maintain
- consistency. They create relationships between different tables.



Keys in DBMS

Importance of Keys in DBMS

Keys are important in a Database Management System (DBMS) for several reasons:

- **Uniqueness:** Keys ensure that each record in a table is unique and can be identified distinctly. • **Data Integrity:** Keys prevent data duplication and maintain the consistency of the data.
- **Efficient Data Retrieval:** Keys help in creating relationships between tables, allowing faster queries and better data organization.

Without keys, managing large databases would become difficult, and data retrieval would be slow and error-prone.

Types of Database Keys

1. ## Super Key

   The set of one or more attributes (columns) that can uniquely identify a tuple (record) is known as Super Key. It may include extra attributes that aren't important for uniqueness but still uniquely identify the row. For Example, STUD_NO, (STUD_NO, STUD_NAME), etc.

identify the row. For Example, STUD_NO, (STUD_NO, STUD_NAME), etc.

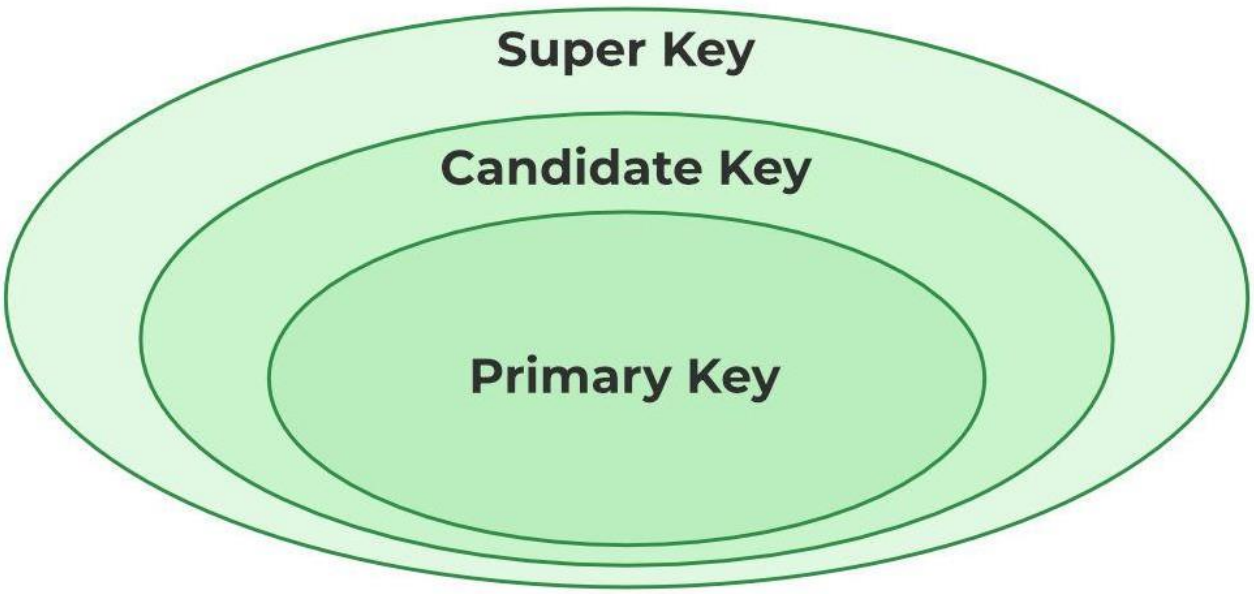
- A super key is a group of single or multiple keys that uniquely identifies rows in a table. It supports NULL values in rows.
- A super key can contain extra attributes that aren't necessary for uniqueness.
- For example, if the "STUD_NO" column can uniquely identify a student, adding "SNAME" to it will still form a valid super key, though it's unnecessary.

Example: Consider the STUDENT table

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

A super key could be a combination of STUD_NO and PHONE, as this combination uniquely identifies a student.



Relation between Primary Key, Candidate Key, and Super Key

2. Candidate Key

The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. For Example, STUD_NO in STUDENT relation.

- A candidate key is a minimal super key, meaning it can uniquely identify a record but contains no extra attributes.
- It is a super key with no repeated data is called a candidate
- key. The minimal set of attributes that can uniquely identify

a record.

- A candidate key must contain unique values, ensuring that no two rows have the same value in the candidate key’s columns.
- Every table must have at least a single candidate key.

- A table can have multiple candidate keys but only one primary key.

Example: For the STUDENT table below, STUD_NO can be a candidate key, as it uniquely identifies each record.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

Table: STUDENT_COURSE

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

STUDENT_COURSE Table

A composite candidate key example: {STUD_NO, COURSE_NO} can be a candidate key for a STUDENT_COURSE table.

3. Primary Key

A primary key is chosen from the set of candidate keys to uniquely identify each record in a table. For example, in the STUDENT table, both STUD_NO and STUD_PHONE can be candidate keys, but STUD_NO is selected as the primary key.

- It uniquely identifies every tuple (row) and does not allow duplicate values.
- It cannot be NULL, as each record must have a valid identifier.
- It may be single-column or composite (made of multiple columns).
- Databases often organize data using the primary key to allow faster access and searching.

Example: The STUDENT table has the structure Student(STUD_NO, SNAME, ADDRESS, PHONE), where STUD_NO is the primary key.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

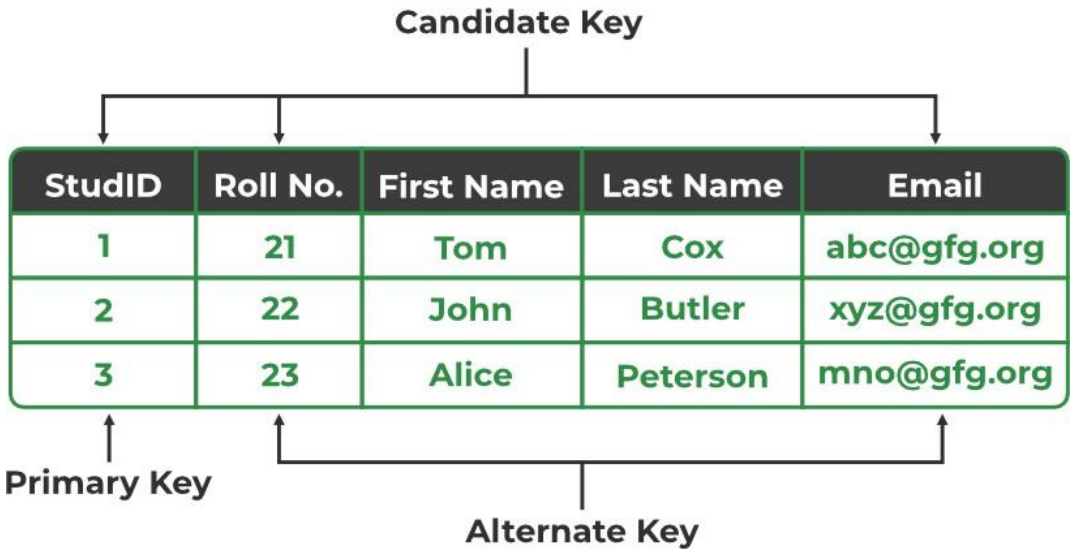
4. Alternate Key

An alternate key is any candidate key in a table that is not chosen as the primary key. In other words, all the keys that are not selected as the primary key are considered alternate keys.

- An alternate key is also referred to as a secondary key because it can uniquely identify records in a table, just like the primary key.

- An alternate key can consist of one or more columns (fields) that can uniquely identify a record, but it is not the primary key

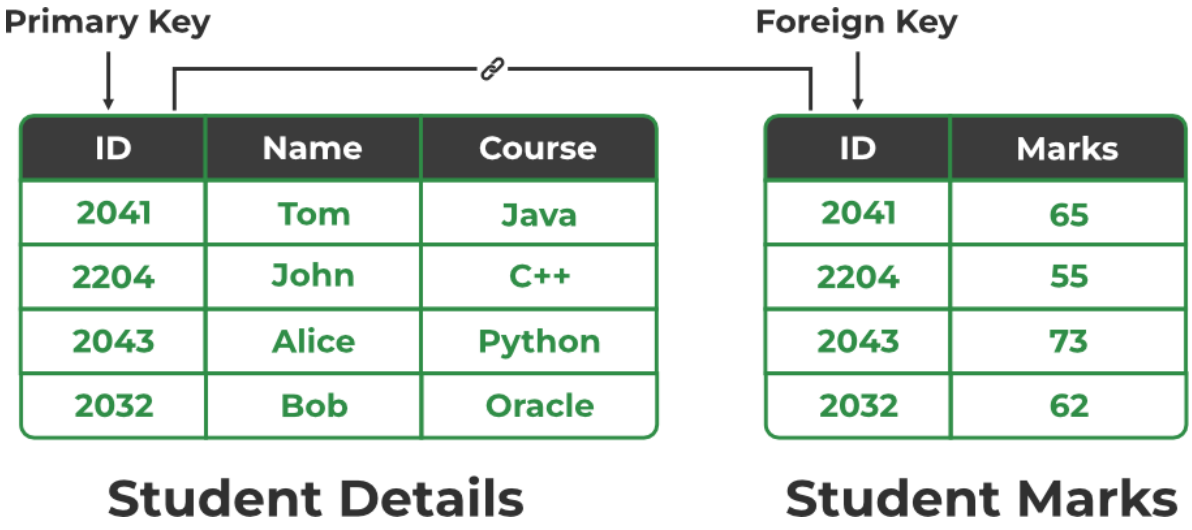
Example: In the STUDENT table, both STUD_NO and PHONE are candidate keys. If STUD_NO is chosen as the primary key, then PHONE would be considered an alternate key.



Primary Key, Candidate Key, and Alternate Key

5. Foreign Key

A foreign key is an attribute in one table that refers to the primary key in another table. The table that contains the foreign key is called the referencing table and the table that is referenced is called the referenced table.



- A foreign key in one table points to the primary key in another table, establishing a relationship between them.

- It helps connect two or more tables, enabling you to create relationships between them. This is important for maintaining data integrity and preventing data redundancy.
- They act as a cross-reference between the tables.

Example: Consider the STUDENT_COURSE table

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

STUDENT_COURSE Table

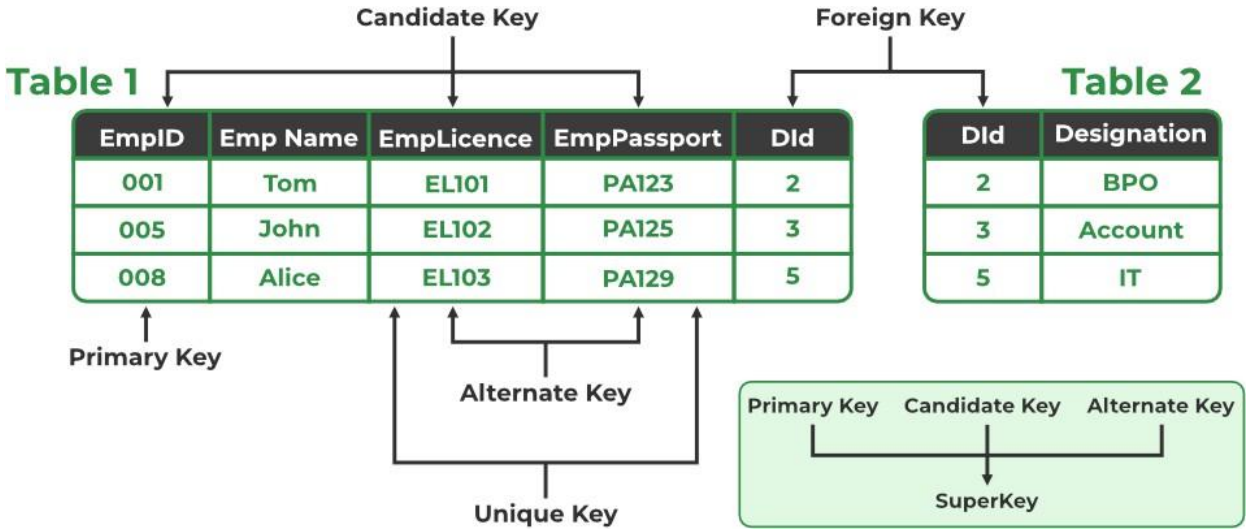
- STUD_NO in the STUDENT_COURSE table is a foreign key that refers to the STUD_NO primary key of the STUDENT table.
- Unlike a primary key, a foreign key can contain duplicate values and may be NULL. For example, STUD_NO appears multiple times in STUDENT_COURSE because a student can enroll in more than one course.
- However, STUD_NO in the STUDENT table is a primary key, so it must always be unique and non- NULL.

6. Composite Key

Sometimes, a single column is not enough to uniquely identify all records in a table, so a combination of multiple attributes is used. An optimal set of such attributes is chosen to ensure that every row is uniquely identifiable.

- It acts as a primary key if there is no primary key in a table
- Two or more attributes are used together to make a composite key .
- Different combinations of attributes may give different accuracy in terms of identifying the rows uniquely.

Example: In the STUDENT_COURSE table, {STUD_NO, COURSE_NO} can form a composite key to uniquely identify each record.


Candidate Key
Foreign Key
Table 1
Table 2
EmpID
Emp Name
EmpLicence
EmpPassport
DId
001
Tom
EL101
PA123
2
005
John
EL102
PA125
3
008
Alice
EL103
PA129
5
DId
Designation
2
BPO
3
Account
5
IT
Primary Key
Alternate Key
Unique Key
Primary Key
Candidate Key
Alternate Key
SuperKey

SQL(Structured Query Language)

Overview

- Introduction
- DDL Commands
- DML Commands
- SQL Statements, Operators, Clauses
- Aggregate Functions

Difference between DBMS and RDBMS

S.N	DBMS	TDBMS
1	Introduced in 1960s.	Introduced in 1970s.
2	During introduction it followed the navigational modes (Navigational DBMS) for data storage and fetching.	This model uses relationship between tables using primary keys, foreign keys .
3	Data fetching is slower for complex and large amount of data.	Comparatively faster because of its relational model.
4	Used for applications using small amount of data.	Used for complex and large amount of data.
5	Data Redundancy is common in this model	Keys and indexes are used in the tables to avoid redundancy.
6	Example systems are dBase,, LibreOffice Base, FoxPro.	Example systems are SQL Server, Oracle ,MS ACESS, MySQL, MariaDB, SQLite.

Structured Query Language (SQL)

- The ANSI standard language for the definition and manipulation of relational database.

- Includes data definition language (DDL), statements that specify and modify database schemas.

- Includes a data manipulation language (DML), statements that manipulate database content.

Some Facts on SQL

- SQL data is case-sensitive, SQL commands are not.
- First Version was developed at IBM by Donald D. Chamberlin and Raymond F. Boyce. [SQL]
- Developed using Dr. E.F. Codd's paper, “A Relational Model of Data for Large Shared Data Banks.”
- SQL query includes references to tuples variables and the attributes of those variables

SQL: DDL Commands

- CREATE TABLE: used to create a table.
- ALTER TABLE: modifies a table after it was created.
- DROP TABLE: removes a table from a database.

- Truncate: use to remove all data
- Rename: use to change table name

SQL: CREATE TABLE Statement

- Things to consider before you create your table are:
  - The type of data
  - the table name
  - what column(s) will make up the primary key
  - the names of the columns

- CREATE TABLE statement syntax:

```
CREATE TABLE <table name>
( field1 datatype ( NOT NULL ),
  field2 datatype ( NOT NULL )
);
```

SQL: Attributes Types

Numeric types	integer	`integer, int, smallint, long`
	floating point	`float, real, double precision`
	formatted	`decimal(i,j), dec(i,j)`
Character-string types	fixed length	`char(n), character(n)`
	varying length	`varchar(n), char varying(n), character varying(n)`
Bit-string types	fixed length	`bit(n)`
	varying length	`bit varying(n)`
Date and time types		`date, time, datetime, timestamp, time with time zone, interval`
Large types	character	`long varchar(n), clob, text`
	binary	`blob`

SQL: ALTER TABLE Statement

- To add or drop columns on existing tables.
- ALTER TABLE statement syntax:

ALTER TABLE <table name>

ADD attr datatype;

or

DROP COLUMN attr;

SQL: DROP TABLE Statement

DROP TABLE statement syntax:

DROP TABLE <table name> ;

Example:

```
CREATE TABLE FoodCart (
date varchar(10),
food varchar(20),
profit float
);
```

FoodCart

date	food	profit

```
ALTER TABLE FoodCart (
ADD sold int
);
```

FoodCart

date	food	profit	sold

```
ALTER TABLE FoodCart(
DROP COLUMN profit
);
```

FoodCart

date	food	sold

```
DROP TABLE FoodCart;
```

SQL: RENAME TABLE Statement & TRUNCATE TABLE Statement

RENAME old_table_name TO new_table_name RENAME Students TO customers

TRUNCATE TABLE table_name TRUNCATE TABLE students

SQL: DML Commands

- INSERT: adds new rows to a table.
- UPDATE: modifies one or more attributes.
- DELETE: deletes one or more rows from a table.

SQL: INSERT Statement

- To insert a row into a table, it is necessary to have a value for each attribute, and order matters.
- INSERT statement syntax:

INSERT into <table name>
VALUES ('value1', 'value2', NULL);

Example: INSERT into FoodCart
VALUES ('02/26/08', 'pizza', 70);

FoodCart

date	food	sold
02/25/08	pizza	350
02/26/08	hotdog	500

date	food	sold
02/25/08	pizza	350
02/26/08	hotdog	500
02/26/08	pizza	70

SQL: UPDATE Statement

- To update the content of the table:

UPDATE statement syntax:

UPDATE <table name> SET <attr> = <value>

WHERE <selection condition>;

Example: UPDATE FoodCart SET sold = 349

WHERE date = '02/25/08' AND food = 'pizza';

FoodCart

date	food	sold
02/25/08	pizza	350
02/26/08	hotdog	500
02/26/08	pizza	70

date	food	sold
02/25/08	pizza	349
02/26/08	hotdog	500
02/26/08	pizza	70

SQL: DELETE Statement

- To delete rows from the table:

DELETE statement syntax:
DELETE FROM <table name>
WHERE <condition>;
Example: DELETE FROM FoodCart
WHERE food = 'hotdog';

FoodCart

date	food	sold
02/25/08	pizza	349
02/26/08	hotdog	500
02/26/08	pizza	70

date	food	sold
02/25/08	pizza	349
02/26/08	pizza	70

Note: If the WHERE clause is omitted all rows of data are deleted from the table.

SQL Statements, Operations, Clauses

- SQL Statements:
  - Select
- SQL Operations:
  - Join
  - Left Join
  - Right Join
  - Like
- SQL Clauses:
  - Order By
  - Group By
  - Having

SQL: SELECT Statement

- A basic SELECT statement includes 3 clauses

SELECT <attribute name> FROM <tables> WHERE <condition>

SELECT	FROM	WHERE
Specifies the attributes that are part of the resulting relation	Specifies the tables that serve as the input to the statement	Specifies the selection condition, including the join condition.

SQL: SELECT Statement (cont.)

- Using a "*" in a select statement indicates that every attribute of the input table is to be selected.

Example: SELECT * FROM … WHERE …;

- To get unique rows, type the keyword DISTINCT after SELECT.

Example: SELECT **DISTINCT** * FROM …
WHERE …;

Example:

Person

Name	Age	Weight
Harry	34	80
Sally	28	64
George	29	70
Helena	54	54
Peter	34	80

1) SELECT *
FROM person
WHERE age > 30;

Name	Age	Weight
Harry	34	80
Helena	54	54
Peter	34	80

2) SELECT weight
FROM person
WHERE age > 30;

Weight
80
54
80

3) SELECT **distinct** weight
FROM person
WHERE age > 30;

Weight
80
54

SQL: Join operation

- A join can be specified in the FROM clause which list the two input relations and the WHERE clause which lists the join condition.

  Example:

Emp

ID	State
1000	CA
1001	MA
1002	TN

Dept

ID	Division
1001	IT
1002	Sales
1003	Biotech

SQL: Join operation (cont.)

- inner join = join

  SELECT *

  FROM emp join dept (or FROM emp, dept)

  on emp.id = dept.id;

Emp.ID	Emp.State	Dept.ID	Dept.Division
1001	MA	1001	IT
1002	TN	1002	Sales

SQL: Join operation (cont.)

- left outer join = left join

  SELECT *

  FROM emp left join dept

  on emp.id = dept.id;

Emp.ID	Emp.State	Dept.ID	Dept.Division
1000	CA	null	null
1001	MA	1001	IT
1002	TN	1002	Sales

SQL: Join operation (cont.)

- right outer join = right join

  SELECT *

  FROM emp right join dept

  on emp.id = dept.id;

Emp.ID	Emp.State	Dept.ID	Dept.Division
1001	MA	1001	IT
1002	TN	1002	Sales
null	null	1003	Biotech

SQL: Like operation

Pattern matching selection

- % (arbitrary string)

  SELECT *

  FROM emp

  WHERE ID like '%01';

  □ finds ID that ends with 01, e.g. 1001, 2001, etc

- _ (a single character)

  SELECT *

  FROM emp

  WHERE ID like '_01_';

  □ finds ID that has the second and third character as 01, e.g. 1010, 1011, 1012, 1013, etc

SQL: The ORDER BY Clause

Ordered result selection

- desc (descending order)

  SELECT *

  FROM emp

  order by state desc

  □ puts state in descending order, e.g. TN, MA, CA
- asc (ascending order)

  SELECT *

  FROM emp

  order by id asc

  □ puts ID in ascending order, e.g. 1001, 1002, 1003

SQL: The GROUP BY Clause

- The function to divide the tuples into groups and returns an aggregate for each group.
- Usually, it is an aggregate function's companion

```
SELECT food, sum(sold) as totalSold
FROM FoodCart
group by food;
```

FoodCart

date	food	sold
02/25/08	pizza	349
02/26/08	hotdog	500
02/26/08	pizza	70

food	totalSold
hotdog	500
pizza	419

SQL: The HAVING Clause

- The substitute of WHERE for aggregate functions
- Usually, it is an aggregate function's companion

```
SELECT food, sum(sold) as totalSold
FROM FoodCart
group by food
having sum(sold) > 450;
```

FoodCart

date	food	sold
02/25/08	pizza	349
02/26/08	hotdog	500
02/26/08	pizza	70

food	totalSold
hotdog	500

SQL: Aggregate Functions

Are used to provide summarization information for SQL statements, which return a single value.

- COUNT(attr)
- SUM(attr)
- MAX(attr)
- MIN(attr)
- AVG(attr)

Note: when using aggregate functions, NULL values are not considered, except in COUNT(*) .

SQL: Aggregate Functions (cont.)

FoodCart

date	food	sold
02/25/08	pizza	349
02/26/08	hotdog	500
02/26/08	pizza	70

- COUNT(attr) -> return # of rows that are not null

Ex: COUNT(distinct food) from FoodCart; -> 2

- SUM(attr) -> return the sum of values in the attr

Ex: SUM(sold) from FoodCart; -> 919

- MAX(attr) -> return the highest value from the attr

Ex: MAX(sold) from FoodCart; -> 500

SQL: Aggregate Functions (cont.)

FoodCart

date	food	sold
02/25/08	pizza	349
02/26/08	hotdog	500
02/26/08	pizza	70

- MIN(attr) -> return the lowest value from the attr

Ex: MIN(sold) from FoodCart; -> 70

- AVG(attr) -> return the average value from the attr

Ex: AVG(sold) from FoodCart; -> 306.33

Note: value is rounded to the precision of the datatype